

Trees in Java

Mostafa Faik

October 13, 2024

Introduction

Binary trees are widely used in computer science due to their efficiency in storing and retrieving hierarchical data. They allow fast search, insertion, and traversal operations, making them ideal for various applications, such as databases and file systems. In this assignment, we worked through different tasks to understand **binary tree** construction, recursive and iterative methods for adding and searching values, various tree traversal techniques, and benchmarking the performance of different implementations. Additionally, we explored the use of **explicit stacks** in tree traversals. This report outlines each task, what we did, and the insights we gained along the way.

Adding Nodes to a Binary Tree

The first step was to create a **binary tree** where each **node** contains an integer value and pointers to its left and right **nodes** (**children**). This structure ensures that for each **node**, values smaller than the **node** are placed on the left, while values greater than the **node** are placed on the right. This makes searching the tree faster. We implemented two ways of adding **nodes** to the tree:

- **Recursive Add:** This method starts from the **root** and compares the value to be added with the current **node's** value. If the value is smaller, it moves to the left **subtree**; if it's larger, it moves to the right. This process continues until a *null* spot is found, where the new **node** is added.
- **Iterative Add:** Instead of using **recursion**, we used a loop to traverse the **tree** and find the correct position to insert the new value. This avoids the overhead of **recursive** function calls, particularly beneficial for larger **trees** where deep recursion could cause a **stack overflow**.

The **recursive approach** is intuitive since it reflects the hierarchical nature of **trees**. However, **recursion** can lead to performance issues when

the **tree** grows large. In contrast, the **iterative** method is slightly more efficient in terms of memory usage, especially for large, unbalanced **trees**.

If we constructed the **binary tree** using an ordered sequence of values, the **tree** would degenerate into a **linked list**, significantly degrading performance due to the lack of balance.

Searching for Values in the Binary Tree

After adding **nodes**, the next task was to implement a **lookup** method to search for a specific value in the **tree**. We implemented both **recursive** and **iterative** versions:

- **Recursive Lookup**: This method works by **recursively** traversing the **tree**, comparing the target value with the current **node**. It moves left or right based on whether the target is smaller or larger than the current **node's** value.
- **Iterative Lookup**: Similar to the **iterative** add method, this approach avoids **recursion**. Instead, it uses a loop to navigate the **tree**, checking each **node** until it either finds the value or reaches a **leaf**.

Both methods are efficient for balanced **trees**, with a time complexity of $O(\log n)$. However, when the **tree** becomes unbalanced (i.e., behaves like a **linked list**), the time complexity can degrade to $O(n)$. While the **recursive** method is easier to implement, the **iterative** method is generally more reliable for large or deeply nested **trees**, as it avoids potential **stack overflow** issues.

Benchmarking Recursive vs. Iterative Methods

We conducted a series of **benchmarks** to compare the performance of the **recursive** and **iterative** versions of the **add** and **lookup** methods. The goal was to determine how the methods performed as the **tree** grew in size. The following table shows the results of the **benchmarks**:

Tree Size	Recursive add (ns)	Iterative add (ns)	Recursive lookup (ns)	Iterative lookup (ns)
100	10.50	5.06	1.45	0.08
200	15.00	8.46	2.06	0.12
400	20.53	12.52	2.87	0.17
800	25.04	15.78	3.23	0.22
1600	31.42	22.46	3.92	0.28
3200	40.74	31.83	4.51	0.34

Table 1: Benchmark Times for Recursive and Iterative Methods

From the results, we observe that the **iterative** methods generally perform faster than their **recursive** counterparts, particularly for smaller input sizes. As the **tree** grew larger, the gap between the two approaches narrowed, but the **iterative** method still maintained a performance edge. This reinforces the idea that **iterative** methods are often more efficient in terms of both speed and memory usage, especially for large datasets.

When the **tree** is constructed with an ordered sequence of values, it degenerates into a **linked list**. This significantly degrades the performance of both `add()` and `lookup()` operations because they now behave like **linear searches**, making them $O(n)$ instead of $O(\log n)$.

In terms of performance, **binary search** is $O(\log n)$ in an **array**, while **binary tree lookup()** is also $O(\log n)$ in a **balanced tree**. However, in an **unbalanced tree** (like a degenerate **tree**), `lookup()` can degrade to $O(n)$, making **binary search** more efficient in such cases.

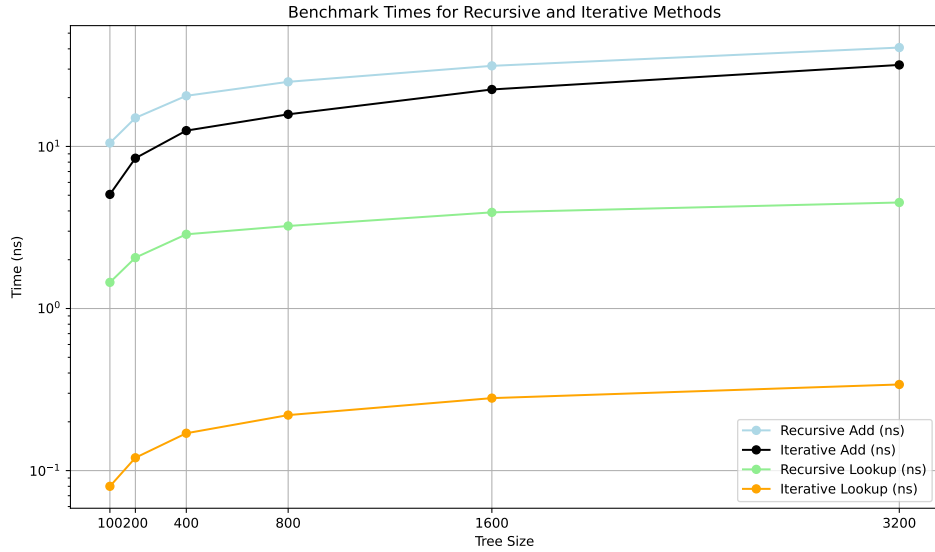


Figure 1: Array Queue Graph Result

Operation	Best Case	Worst Case
Recursive add()	$O(\log n)$ (balanced tree)	$O(n)$ (unbalanced tree)
Iterative add()	$O(\log n)$ (balanced tree)	$O(n)$ (unbalanced tree)
Recursive lookup()	$O(\log n)$ (balanced tree)	$O(n)$ (unbalanced tree)
Iterative lookup()	$O(\log n)$ (balanced tree)	$O(n)$ (unbalanced tree)

Table 2: Expected Time Complexity for Recursive and Iterative Methods

Comparing Recursive and Iterative Versions Performance

The `recursive` and `iterative` versions should show similar time complexity because they both traverse the `tree`. However, `iterative methods` may avoid the overhead of `recursion` (such as function call overhead), making them slightly faster for very large inputs. This difference will usually become noticeable with larger tree sizes.

The expected output will show that the time taken for `add()` and `lookup()` grows as the `tree` size increases, and if the `tree` stays balanced, the growth will be logarithmic $O(\log n)$. However, in cases where the `tree` becomes unbalanced, the time complexity will approach linear $O(n)$.

Ordered Insertion (Degenerate Case)

If we were to insert elements in a sorted order (e.g., 1, 2, 3, 4, 5...), the `tree` will become highly unbalanced, resembling a `linked list`. In this case, the `add()` and `lookup()` operations will perform closer to $O(n)$.

In contrast, if elements are inserted in a random order, the `tree` will likely stay closer to balanced, and we should observe performance close to $O(\log n)$ for both operations.

Depth-First Traversal

We explored three types of `depth-first traversals`:

- In-Order Traversal: This traversal visits `nodes` in ascending order. It first processes the left `subtree`, then the current `node`, and finally the right `subtree`.
- Pre-Order Traversal: This approach processes the current `node` first, followed by the left and right `subtrees`.
- Post-Order Traversal: Here, we process the left and right `subtrees` before visiting the current `node`.

We implemented these traversals using `recursion`, which naturally fits the structure of the `binary tree`. This was a straightforward task since recursion automatically manages the traversal state.

`In-order traversal` is particularly useful for `binary search trees` because it processes `nodes` in ascending order. `Pre-order` and `post-order` traversals have their own use cases, such as creating copies of `trees` or evaluating expressions.

Implementing an Explicit Stack for DFS

We adapted a `dynamic stack` implementation from a previous assignment to explicitly manage the traversal of the `binary tree`. This `explicit stack` stores the `nodes` to be processed and simulates the `in-order traversal`. The challenge here was to maintain an invariant, ensuring that the left `subtree` of any popped `node` has already been processed, while the `node` itself and its right `subtree` have not.

Using this `explicit stack` allows more control over the traversal process and is especially useful in situations where the `stack's state` needs to be saved or manipulated, unlike the `implicit stack` which is discarded after the `recursion` completes.

Implementing an `explicit stack` requires careful management of the `traversal state`. While it is more complex to implement compared to using `recursion`, this approach can be advantageous in certain scenarios where control over the traversal process is necessary.

Conclusion

Through this assignment, we explored the intricacies of `binary tree` construction, `node` insertion, value searching, and `depth-first traversal`. The `recursive` and `iterative` methods offer different trade-offs in terms of simplicity and performance, with `iterative` methods proving to be more efficient for larger datasets. The use of an `explicit stack` for `tree traversal`, while more complex, allows for greater control over the traversal process and can be useful in specific scenarios.

The `benchmarks` clearly demonstrated that `iterative` methods outperform `recursive` ones as the `tree` size increases, highlighting the importance of choosing the right implementation for efficiency, especially when handling large or `unbalanced trees`. Overall, this assignment provided valuable insights into the design and performance considerations involved in working with `binary trees`.